

# On a Low Cost Fault Injection Framework for Security Assessment of Cyber-Physical Systems: Clock Glitch Attacks

Zahra Kazemi<sup>1</sup>, Athanasios Papadimitriou<sup>2</sup>, Ioanna Souvatzoglou<sup>1</sup>, Ehsan Aerabi<sup>1</sup>, Mosabbah Mushir Ahmed<sup>1</sup>, David Hely<sup>1</sup>, Vincent Beroulle<sup>1</sup>

<sup>1</sup>Univ. Grenoble Alpes, Grenoble INP, LCIS, F-26000 Valence, France

<sup>2</sup>Univ. Grenoble Alpes, Grenoble INP ESISAR, ESYNOV, F-26000 Valence, France

**Abstract**— Fault injection methods as a type of physical attack have gained significant importance in the security of MCU-based Internet-of-Things (IoT) systems and they continue to become more and more important as the value of assets continues to increase. These attacks can pose severe security risks to the entire IoT system and their effects can quickly lead to security breaches. However, embedded software developers most often do not have the necessary expertise concerning existing vulnerabilities against such attacks. This makes it necessary to have a practical evaluation platform for measuring the degree of security, in a rapid and accurate way. These platforms are important not only from the performance and capability point of view, but also they need to be cost-effective, which is a critical factor to consider during their design. We present in this work a generic low cost and open platform, called HackMyMCU framework. While this platform offers both side channel and fault injection capabilities, this paper focuses on its clock glitch attacks. A first review of existing clock-based fault injectors is initially performed. Then we present two different clock glitchers and we use them to evaluate a modern MCU. The suggested methods consider the necessary parameters for the development of cost-effective and easy to use evaluation platforms which utilize easily accessible equipment. The findings show that a common and low-cost evaluation platform can be implemented with the goal to validate appropriate countermeasures against such attacks.

**Keywords**—Hardware Security, Fault Injection, Clock Tampering

## I. INTRODUCTION

The rapid growing process of pervasive connected embedded devices has provided today's world with the IoT concept, along with so many benefits for the industry and the society. In recent years, these devices have gained a significant importance in many aspects, ranging from safety critical applications to consumer electronic devices, such as medical IoT devices [1], [2]. Along with the diverse opportunities that IoT devices provide, they also present various challenges. In addition to demanding performance and power management challenges, security vulnerabilities and breaches are important to be considered. Regarding security, embedded software developers have to protect their applications against numerous network and software-based attacks and apply different protection protocols (e.g. secure authentications, light weight cryptography algorithms, digital signatures) to secure their designs against vulnerabilities. Considering the fact that these devices may be deployed in uncontrolled, complex, and sometimes hostile environments which can be easily accessible

by adversaries, the existing vulnerabilities against physical attacks become a new challenge. For instance, medical IoT devices, such as connected infusion pumps, which are used to deliver medicine to patients remotely have been the target of attackers [3]. Broad adoption and the vital role of such devices show the need for safe functionality as well as data security and confidentiality [2], [3].

There exist various groups of physical attacks and they are divided into two main classes: 1) *passive attacks* and 2) *active attacks*. One important type of the first category is *side-channel attacks (SCA)*. In such attacks the execution time, power consumption and electromagnetic emanations are three parameters the attacker can use to leak information and exploit to reveal secret data [2], [4]. Active attacks as the second kind of physical attacks aim at causing malfunctions on the targeted devices by modifying the operating environment. *Fault injection attacks (FI)* can be performed by an adversary to either force the device to bypass security mechanisms or to extract secret information by using faulty outputs. These attacks have demonstrated to be a powerful technique to corrupt the data flow of a system, to hijack its control flow or to reveal secret data with a small number of experiments [5]. The common ways of performing the FI at a reasonable cost is done by manipulating the external clock or power inputs or by using electromagnetic disturbances. Some of these methods can be applied by a single motivated attacker with mid-level expertise and low-cost equipment, and thus, these fault injection techniques should be considered as a serious threat to IoT systems [6]. Generally, these attacks are not well explored by general purpose IoT developers and the attacker can easily override types of protections and encryptions implemented without taking such attacks into account. Therefore, it is necessary to provide a comprehensive platform to developers, which integrates all the requirements so as to evaluate their designs against physical attacks. Evaluation of the manufactured devices against physical attacks is usually performed by mounting different types of known attacks using various platforms [7]–[10]. The complexity and difficulty of comparing the results of evaluations on different platforms have been discussed in [9], [10]. Recently, various evaluation platforms such as SAKURA [8] and SASEBO [9], have been proposed. They form efficient tools to evaluate the resilience of a given FPGA-based cryptographic implementations against side channel and can also be used to perform fault injection attacks. However, these platforms were not developed to be used to evaluate general purpose microcontroller based

systems. They are generally applicable for complex designs built for FPGAs or to evaluate RTL code destined for customized ASICs or smart cards. Authors in [10] Introduce *Chipwhisperer*® as a low cost and open evaluation platform for side-channel analysis and fault attacks by glitching techniques, which consists of hardware and software to implement the evaluations. This platform contains target devices, measurement tools and attack software.

Improving the security of the products can prevent hardware threats and protects important assets. Next generation of evaluation platforms need to be low cost, so that they can be broadly used for the validation of secure applications by academia and industry, without the need to purchase specialized expensive hardware. Inspired of the success of open-source software in [10], open-source hardware design of such evaluation platforms is a way to share innovation in vulnerability evaluation and improve security levels. Such an approach also makes evaluation platforms available to a broader audience and can lead to a faster development of new methods. In [2] we have introduced the goals and first steps on developing a low cost and open platform, named “HackMyMCU” to assist software developers to evaluate the resilience of their designs against side channel and fault attacks. This work is performed in the context of IoT for medical applications, which requires proper security mechanisms. However, our ultimate goal is to provide a complete and open source evaluation platform, and encourage hardware security specialists to use and build on that, in order to evaluate their applications. This paper presents an extension of this platform. In this work we present two different implementations of clock glitch generators to perform fault injection evaluations. Then, we show that by using these fault injection tools, we are able to target specific computations of a software implementation of AES and successfully inject exploitable faults. The proposed platform will ultimately target clock, power and electromagnetic attacks. Next steps of this work will target the retrieval of the secret encryption key as well as conditional executions in authentication protocols.

This paper is organized as follows: section II reviews the existing clock glitch fault generators and discusses the related parameters. Section III presents the evaluation platforms we have designed to apply practical fault injection campaigns. Section IV is devoted to the preliminary results obtained by using the platforms. Finally conclusions and perspectives are presented in section V.

## II. CLOCK GLITCHING FAULT INJECTORS

### A. Fault Injection Basic Set-Up

Fig.1 describes the general experimental set-up for the hardware-controlled fault injections. It includes a target board, a control computer and a fault generator. The generator is set to perform the physical attack on the target and to induce faults into the running application. Also, the control computer is used to get the desired results and to obtain higher success rate by generating well-controlled faulty signals by the attacker. The synchronization of the aforementioned modules is provided through various communication ports. Noting that, improving some parameters, such as the cost, the complexity of the set-up,

the required user expertise, etc., can improve the efficiency and practicality of the fault injection set-up.

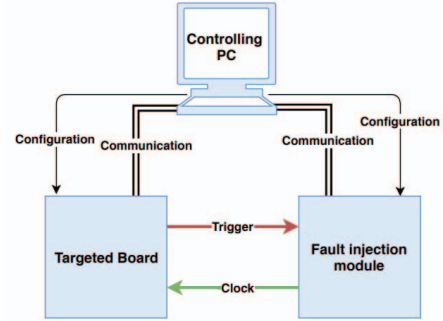


Fig. 1. General Fault Injection Attack Set-up

### B. Fault Attack By Tampering The Clock Input

Clock fault attacks are based on creating erroneous behavior in targets by violating the circuit’s timing rules [11]. In digital systems, for normal and safe operation, a dependable and steady clock frequency is needed. Tampering the clock signal of these systems, either continuously (*overclocking*) or instantly (*clock glitch*), can be applied by the attacker in order to inject faults. *Overclocking* aims at feeding the target circuit with a clock frequency which is higher than the maximum one that the circuit can support. A *Clock glitch* is any sudden variation in the input clock frequency of a device under evaluation. If the abnormal clock period during clock glitching is called  $T_{Glitch}$ , in order to have an erroneous behavior,  $T_{Glitch}$  should be less than  $T_{min}$ , which is equal to the reciprocal of maximum frequency[11].

Various methods have been implemented to generate and induce glitches into the target’s clock signal [8], [10]–[15]. In the following, the most important and practical techniques have been reviewed. Fig.2 illustrates the first method. It shows that by switching between two different clock signals with slightly different phases which are created from the main clock, we can generate a glitch stream signal [11]. Switching between signals with different frequencies is another approach to create the clock glitches and it has been shown in Fig.3 [12]. Finally, we are able to insert the glitches into the main clock signal on a specific predefined time (based on selection signal) and obtain the faulty clock.

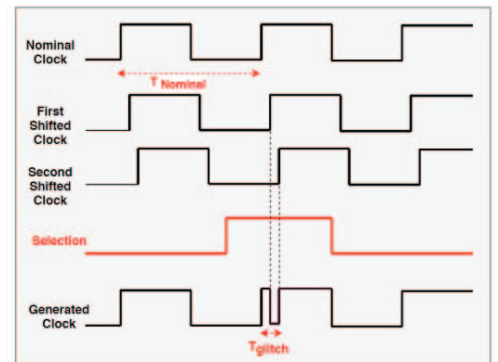


Fig. 2. Faulty clock generated by combining phase shifted clocks

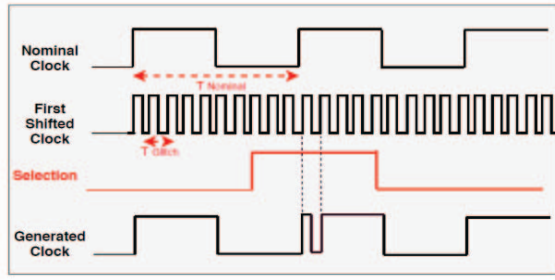


Fig. 3. Faulty clock generated by combining clocks with different frequencies

There can be two methods to divide and shift the nominal clocks and further combine them. In [11], authors use the embedded Delayed locked loop (DLL) of a FPGA in order to generate the shifted clock signals. The same method is applied in [8], [14] and [16] with respectively the SAKURA and SASEBO platforms. In [12] a similar method is considered, in which the output clock is made by switching between three signals with equal periods and different phases. Chipwhisperer® [10] uses two variable phase-shift modules like many previous works to produce a glitch stream. In [17] researchers apply this method by using a 2-to-1 multiplexer and a two channel pulse generator. In [12], [13], [18] and [19], as the second approach, the clock is made up of a combination of nominal clock and high frequency clock, whose period are  $T_{\text{nominal}}$  and  $T_{\text{glitch}}$ , respectively. Technically, a waveform generator generates an output clock; then by dividing the output clock by a different ratio, the nominal and glitchy clocks were determined.

The advantage of the first mechanism (combining shifted clocks) is that we can have more granularities with a glitch than the second mechanism (combining clocks with different frequencies) [14]. The main reason is that for the clock switch approach the maximum possible clock frequency of an FPGA does not allow to reach a ratio of fast-clock/slow-clock which is equivalent to 360. On the other hand the phase shift approach makes use of the minimum achievable phase shift of  $1/360$  of the clock period. This difference is more evident especially as the target's nominal clock frequency increases.

In case where the attacker aims to exploit some faults by targeting specific instruction or location of the memory, it would be theoretically better to try to get the desired result by adjusting the faulty clock parameters accordingly like all the aforementioned methods. On the contrary, when the goal is just a general testing of a device against clock fault injection and no special vulnerability is considered, this process cannot be accomplished at a reasonable and appropriate time. According to that, the glitch signals presented in [13] are applied with some new concepts to replace the precise glitch with the fuzzy glitch signal. In [13], ring oscillators are used instead of a waveform generator to generate the clock source. In this method the attacker does not need to adjust faulty signal parameters and predict a set combination of them to achieve the desired results and this approach seems to have a better coverage of the whole system.

In next section, we propose two fault generator designs to

induce accurate single glitch into the clock signal. The results show that these faulty clock signals are capable to exploit desired errors in the running application on the target board.

### III. HACKMYMCU CLOCK TAMPERING SETUP

In this section we introduce the detailed architectures of one Phase Shift and one Clock Switch glitch generator. For our setups, we follow the high-level diagram of Fig. 1. The setup consists of a control PC, a target board and a fault injection module. We can control and configure both the fault injection module and the target board by using the PC. The two discussed attack setups are then validated on an off-the-shelf ARM-Cortex-M3 32bit microcontroller (MCU) target. The communication between the MCU and the PC is performed by UART. Moreover, this PC, acting as a user interface configures the both clock glitch generators via a second UART interface.

The target board (MCU in this study) sets the trigger signal when it reaches specific point in execution flow under evaluation. This signal can help us to synchronize the fault injection module and target board. The target board sends the results to the PC to analyze the effects of the injected glitch. In this study, two different methods are proposed to generate and to induce the glitch into the clock signal. The two designs are detailed hereafter. Afterwards, in the result section, their outputs as well as their capabilities and performances are compared.

#### 1) Glitch generation using two variable phase shift modules (Phase-Shift glitch generator)

We follow the method suggested in [15]. The phase shift modules are implemented using the Digital Clock Manager (DCM) of a Xilinx FPGA (Arty-S7-50). In order to inject faults into the FPGA output clock signal, we first generate two phase-shifted clocks from the nominal clock. Then, glitch stream is generated by switching between two signals with the same frequency but different phases (Fig.4). Here, we combine these two signals by applying logical operations as depicted in Fig. 5.

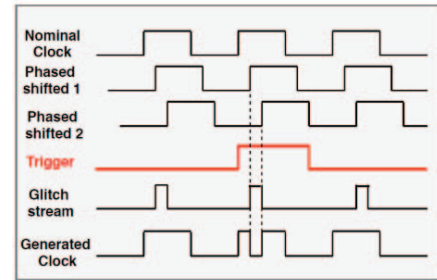


Fig. 4. Clock glitch generation using phase shifted signals

A single glitch can be injected using a trigger signal; once there is a rising edge of the trigger signal, the glitch with the configured parameters can be injected. This signal allows a wide range of glitch injection patterns. In this work, we use a 16-MHz nominal clock generated by the FPGA. We can generate  $T_{\text{glitch}}$  values between 3 ns and 30ns with 174ps step-size. Furthermore, we can control important glitch parameters such as 1) the *Glitch width*, 2) the *Glitch location* inside the affected clock and 3) the *Glitch delay*.



- The glitch width is defined by setting the phase difference of the two shifted clocks. Like in [10], the run-time configuration for this parameter is related to DCM block specifications and partial reconfiguration can make it possible to change this parameter with some restrictions.
- The glitch location inside the affected clock cycle is specified by the phase of the first clock signal. This can have a high impact on the results as we will see in next section.
- The glitch delay is related to the number of clock cycles after the trigger signal and allows us to target clock cycles of the computation of specific machine instructions.

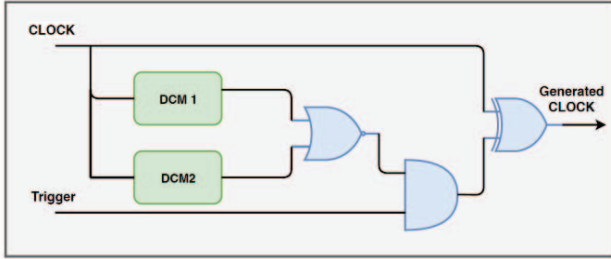


Fig. 5. The block diagram of clock glitch generator circuit

## 2) Glitch generation using switching between two clock sources (Clock-Switch glitch generator)

The second technique was developed to provide a flexible and easy to develop clock glitch platform. It is capable of injecting well controlled single or multiple clock glitches as well as to perform overclocking for specific durations. As shown in Fig. 6, the glitcher is composed by a Kintex 7 FPGA (Digilent Genesys-2) which is controlled by MATLAB.

Inside the FPGA we make use of a clock generator (MMCM) to produce two clocks, with the goal to switch between them to induce a clock glitch as in [12]. The first one (slow clock) is the clock which is provided to the device under evaluation and it is used during its normal operation. For the ARM Cortex core we use as a target in this work a nominal clock of 16MHz. The second one (fast clock) is as fast as possible according to the limitations of the FPGA's dedicated clock output pin's toggling rate for single ended signals. For the FPGA we used in this work the fastest clock which we measured of being output, without affecting its amplitude, was 208MHz. Then the 2 clocks are fed in a specialized clock multiplexer which is available for Xilinx FPGAs (BUFGMUX).

The multiplexer is used in "asynchronous" mode in order to switch from the slow clock to the fast clock, at the time we want to inject the clock glitch. The glitch injection is therefore realized by controlling the select of the multiplexer. This select is connected to a large shift register which is configured by the computer, through a UART connection. Each bit inside the shift register corresponds to one potential clock cycle of the fast clock. Thus when the output of the shift register equals zero, the multiplexer outputs the slow clock, while when it is equal

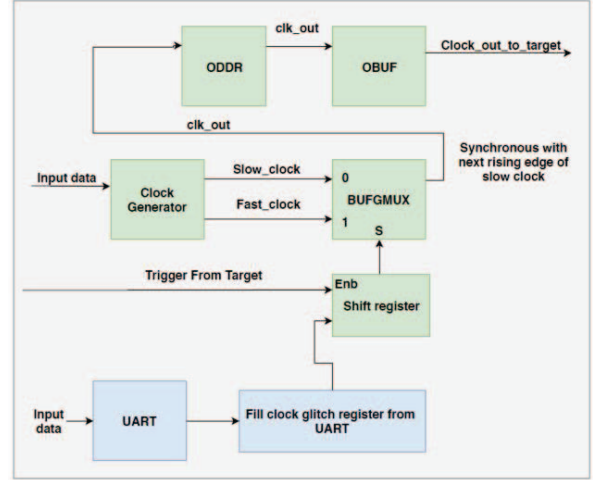


Fig. 6. Clock-Switch glitch generator Architecture

to one, it outputs the fast clock (Fig.7). In order to control the fault injection, we have implemented a state machine in the FPGA, which initially configures the shift register. Then the system is waiting for a trigger from the target and synchronizes the shifting of the register to the select of the BUFGMUX on the "slow clock" edge, following the trigger.

The software controlling the device under evaluation activates the target, which in turn provides a trigger to the FPGA to setup the evaluation interval. After the start of shifting the values, one clock glitch will be applied to the target whenever the output of the shift register equals one. The output of the BUFGMUX is connected with an ODDR and an OBUF element so as to reduce jitter and improve the driving capabilities for the output clock. The methodology used can inject a clock glitch on multiple instants within a normal clock cycle according to the ratio of "fast clock"/"slow clock", which in our case was  $208/16 = 13$ . Therefore, we are able to supply a clock glitch to the target in 13 different time divisions of each clock cycle of the normal clock.

This setup is very flexible since it allows us to configure any combination of single or multiple glitches during the computation under evaluation. The main drawback of the current setup is that if the glitch control shift register is very large, then it takes time if we need to fill it before every fault injection. Nonetheless, it is always possible to alter the FPGA logic in order to provide through the UART, only a pattern of glitches and how they should change before every different fault injection. While theoretically the resulting glitch should consist in a pulse having a period of 4.8ns.

## IV. RESULTS AND DISCUSSIONS

In order to characterize the capabilities of the clock glitch injection platforms, we have performed a fault injection campaign on an AES algorithm running on an ARM-Cortex-M3 32bit MCU. We have attacked the AddRoundKey operation of last round (10<sup>th</sup> round) and on every one of the 410 single glitch which has the same location inside each of the clock edges needed for its computation, we have induced a cycles of the nominal clock, for each glitch generator. The location of the glitch is different between the two generators

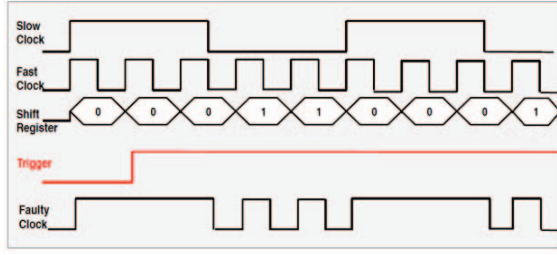


Fig.7 Clock-Shift glitch generation example

and it was determined based on the location which maximized the number of successful fault injections for each of the generators. For both generators we have used exactly the same target. After applying the glitch, we send and save the computed cipher-text of the AES to the computer. The main goal of this experiment is to characterize the types of the injected faults. In this work, 4100 fault injections were performed. Furthermore, in order to be able to verify the behavior of the fault injection with different data being processed, we perform for every different setup of glitch attack parameters, 10 fault injections with random plaintexts. Thus, in each of the 410 clock cycles of the last round AddRoundKey operation we perform 10 fault injections, while all 4100 encryptions are performed with random plaintexts and the same key.

Figures 8 and 9 depict a cartography of the injected faults for the two injectors. The horizontal axes contain all 410 clock cycles of the AES operation under attack. On the vertical axes we show the number of affected bytes of the state register of the AES, due to each of the 410 depicted attacks (with random plaintexts and the same key). When a fault injection resulted to a reset/hang of the MCU we assign to it the value of minus one. Such a cartography is very useful since it can show which are the clock periods of the computation which can lead to a fault of a specific impact. The obtained patterns for the ten repetitions of the attacks were very similar. This shows that there is a consistency during the fault injection with different plaintexts, concerning the number of affected bytes. Such a cartography can be used later so as to focus on a more thorough evaluation of specific clock cycles. Comparing the two figures, we notice that use of the Phase Shift glitch generator led to fewer successful fault injections than the Clock Switch generator, which shows that the overall setup and the glitch location used for fault injection plays an important role for the acquired results.

In table I, we can see the results of all 4100 injections and the corresponding percentages of injections which led either to a fault free operation or to a Hang/Reset of the MCU or to a successful injection, and in that case the number of bytes which were affected. This can confirm reproducibility and also help us to figure out if there is a relation between plain text and number of faulty bytes or not. We notice that the faults which led to a Hang/Reset were 13.4% for the Clock Switch generator, while 25.1% for the Phase Shift generator.

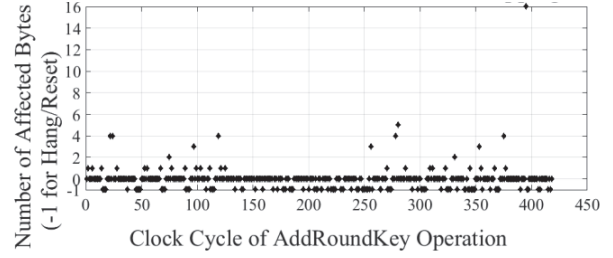


Fig.8. Phase Shift Glitch Generator, Fault Mapping

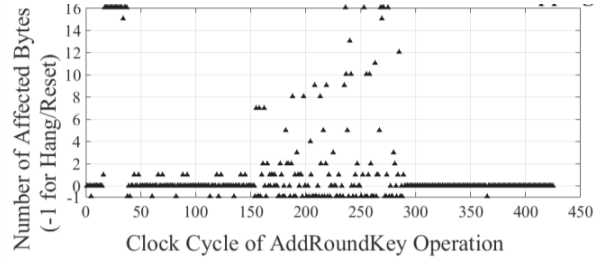


Fig.9 Clock Switch Glitch Generator, Fault Mapping

Furthermore, the Clock Switch glitcher led to considerably more errors affecting one or two bytes and at the same time it caused more faults affecting all 16 bytes of the AES.

In table II we provide results concerning the number of bit-flips which were injected in the subset of injections affecting a single byte of the AES. This time we notice that the Clock Switch glitch generator did not lead to any faults of one bit, while the Phase Shift generator led to 1.2% of errors of a single bit. On the other hand the Clock Switch glitcher led to a higher amount of 255 single byte faults versus 161 for the Phase Shift glitch generator. Even though the results of the two glitchers are different, they show that both glitch generators are capable of injecting well controlled faults in the MCU.

## V. CONCLUSIONS

Two different implementations of a low cost and practical clock glitch fault injectors have been presented. An evaluation of the two modules has been performed with a fault injection campaigns on an MCU running the AddRoundKey operation of the AES algorithm. The results show that the implemented fault injectors can induce precise faults in modern MCU-based systems and can potentially threaten the security of the system. The flexibility to customize and configure it according to the developer needs to inject specific faults can lead to accurate evaluation. While the experiments have been carried out a cryptographic module, further experiments are on-going targeting a medical application in order to identify possible critical weakness in the application. These results show the importance and interest of open source hardware evaluation platform to increase the security level of embedded software.

TABLE I. GLITCH GENERATOR COMPARISON OF AFFECTED BYTES

|                             | Number of affected bytes & Hang/Reset percentages (%) out of 4100 injections |      |     |     |     |     |     |     |     |     |     |     |     |     |     |    |     |    |
|-----------------------------|------------------------------------------------------------------------------|------|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|----|-----|----|
|                             | Hang<br>Reset                                                                | 0    | 1   | 2   | 3   | 4   | 5   | 6   | 7   | 8   | 9   | 10  | 11  | 12  | 13  | 14 | 15  | 16 |
| Phase<br>Shift<br>Glitcher  | 25.1                                                                         | 66.4 | 3.9 | 0.5 | 0.7 | 1.1 | 0.4 | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0  | 0.1 | 2  |
| Clock<br>Switch<br>Glitcher | 13.4                                                                         | 64.5 | 6   | 2.8 | 0.6 | 0.1 | 0.6 | 0.4 | 0.6 | 0.8 | 0.6 | 0.7 | 0.3 | 0.2 | 0.2 | 0  | 0.3 | 8  |

TABLE II. FAULT MULTIPLICITY OF SINGLE BYTE FAULTS

|                                         | Number of affected bits in single byte faults |     |      |      |      |      |      |     |
|-----------------------------------------|-----------------------------------------------|-----|------|------|------|------|------|-----|
|                                         | 1                                             | 2   | 3    | 4    | 5    | 6    | 7    | 8   |
| Glitcher 1<br>(161 total 1-byte faults) | 1.2                                           | 1.2 | 2.5  | 26.1 | 30.4 | 26.1 | 12.4 | 0   |
| Glitcher 2<br>(255 total 1-byte faults) | 0                                             | 11  | 27.1 | 29   | 19.2 | 9    | 4.3  | 0.4 |

## ACKNOWLEDGMENT

This work is carried out under the SERENE-IoT project, a project labeled within the framework of PENTA, the EUREKA cluster for Application and Technology Research in Europe on NanoElectronics.

## REFERENCES

- [1] A. B. Pawar and S. Ghumbre, "A survey on IoT applications, security challenges and counter measures," *Int. Conf. Comput. Anal. Secur. Trends, CAST 2016*, pp. 294–299, 2017.
- [2] Z. Kazemi, A. Papadimitriou, D. Hely, M. Fazeli, and V. Beroulle, "Hardware Security Evaluation Platform for MCU -based Connected Devices: Application to healthcare IoT," *3rd Int. Verif. Secur. Work.*, 2018.
- [3] P. A. H. Williams and A. J. Woodward, "Cybersecurity vulnerabilities in medical devices: A complex environment and multifaceted problem," *Med. Devices Evid. Res.*, vol. 8, pp. 305–316, 2015.
- [4] J. Dubeuf, D. Hely, and V. Beroulle, "ECDSA Passive Attacks, Leakage Sources, and Common Design Mistakes," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 21, no. 2, pp. 1–24, 2016.
- [5] N. Liao, X. Cui, K. Liao, T. Wang, D. Yu, and X. Cui, "Improving DFA attacks on AES with unknown and random faults," vol. 60, no. April, pp. 1–14, 2017.
- [6] R. Piscitelli and F. Regazzoni, "Fault attacks, injection techniques and tools for simulation," pp. 15–20, 2015.
- [7] "Riscure Spider." [Online]. Available: <https://www.riscure.com/product/spider/>.
- [8] M. Matsubayashi, A. Satoh, and J. Ishii, "Clock glitch generator on SAKURA-G for fault injection attack against a cryptographic circuit," *2016 IEEE 5th Glob. Conf. Consum. Electron. GCCE 2016*, pp. 5–8, 2016.
- [9] T. Katashita, Y. Hori, H. Sakane, and A. Satoh, "Side-Channel Attack Standard Evaluation Board SASEBO-W Specification Ver 1.1," *Niat 2011*, p. 36, 2011.
- [10] C. O. Flynn and Z. D. Chen, "chipwhisperer," 2015.
- [11] M. Agoyan, J. M. Dutertre, D. Naccache, B. Robisson, and A. Tria, "When clocks fail: On critical paths and clock faults," *Lect. Notes Comput. Sci. (including Subser. Lect. Notes Artif. Intell. Lect. Notes Bioinformatics)*, vol. 6035 LNCS, pp. 182–193, 2010.
- [12] J. Balasch, B. Gierlichs, and I. Verbauwhede, "An in-depth and black-box characterization of the effects of clock glitches on 8-bit MCUs," *Proc. - 2011 Work. Fault Diagnosis Toler. Cryptogr. FDTC 2011*, pp. 105–114, 2011.
- [13] J. Obermaier, R. Specht, and G. Sigl, "Fuzzy-glitch: A practical ring oscillator based clock glitch attack," *Int. Conf. Appl. Electron.*, 2017.
- [14] S. Endo, T. Sugawara, N. Homma, T. Aoki, and A. Satoh, "An on-chip glitchy-clock generator for testing fault injection attacks," pp. 265–270, 2011.
- [15] B. Yuce, N. F. Ghalaty, and P. Schaumont, "Improving fault attacks on embedded software using RISC pipeline characterization," *Proc. - 2015 Work. Fault Diagnosis Toler. Cryptogr. FDTC 2015*, pp. 97–108, 2016.
- [16] T. Korak and M. Hoefler, "On the effects of clock and power supply tampering on two microcontroller platforms," *Proc. - 2014 Work. Fault Diagnosis Toler. Cryptogr. FDTC 2014*, pp. 8–17, 2014.
- [17] T. Fukunaga and J. Takahashi, "Practical fault attack on a cryptographic LSI with ISO/IEC 18033-3 block ciphers," *Fault Diagnosis Toler. Cryptogr. - Proc. 6th Int. Work. FDTC 2009*, pp. 84–92, 2009.
- [18] J. Korczyk and A. Krasniewski, "Evaluation of Susceptibility of FPGA-based Circuits to Fault Injection Attacks Based on Clock Glitching," pp. 2–5, 2012.
- [19] Y. Qiao, Z. Lu, H. Liu, and Z. Liu, "Clock Glitch Fault Injection Attacks on an FPGA AES Implementation," vol. 1, no. 1, pp. 23–27, 2017.